ENTRAÎNEMENT PRATIQUE • DÉVELOPPEMENT D'APPLICATIONS JAVA / SPRING BOOT

TP Blanc : WS RESTful, Consommation & Spring Security

Architecture distribuée en deux services • Base de données MySQL • Thymeleaf & RestTemplate

> **Contexte et Architecture Générale :** Vous devez concevoir et développer un système de gestion d'inventaire et de logistique composé de **deux applications Spring Boot distinctes** communiquant via HTTP/JSON :
>
> - **Application A (`api-inventario` — Port 8081) :** Serveur backend pur exposant une API RESTful (sans vues HTML ni Thymeleaf) connectée à MySQL.
> - **Application B (`app-logistica` — Port 8080) :** Application web front-end (Spring MVC + Thymeleaf) sécurisée par Spring Security et Spring Session JDBC. Elle n'accède pas aux tables métier directement mais consomme l'API A ainsi qu'une API externe de validation fiscale.

Partie 1 : Base de données & Spring Security (`app-logistica`)

1.1 Modèle Relationnel MySQL (`tp_examen_db`)

Créez la base de données `tp_examen_db` contenant les quatre tables suivantes ainsi qu'un jeu de données initial :

- `rol` (`idrol` PK, `nombre`) — Insérer deux rôles : `admin` et `logistica`.
- `usuario` (`idusuario` PK, `nombre`, `apellido`, `email` UNIQUE, `pwd`, `activo`, `idrol` FK) — Les mots de passe doivent obligatoirement être chiffrés avec **BCrypt** (`BCryptPasswordEncoder`).
- `proveedor` (`idproveedor` PK, `razon_social`, `ruc`, `activo`).
- `producto` (`idproducto` PK, `nombre`, `precio`, `stock`, `idproveedor` FK).

1.2 Configuration de la Sécurité (`WebSecurityConfig`)

Dans `app-logistica`, créez la classe de configuration `WebSecurityConfig` annotée avec `@Configuration` et respectez les contraintes suivantes :

- **Contrôle d'accès par URL (`SecurityFilterChain`) :**
  - Les routes `/`, `/catalogo/**` et `/loginForm` doivent être accessibles sans authentification (`permitAll()`).
  - Les routes `/productos/**` doivent être restreintes aux rôles `admin` et `logistica` (`hasAnyAuthority`).
  - Les routes `/proveedores/**` doivent être accessibles uniquement au rôle `admin` (`hasAuthority`).
- **Authentification JDBC :** Déclarez les beans `PasswordEncoder` (`BCryptPasswordEncoder`) et `UserDetailsManager` (`JdbcUserDetailsManager`). Configurez `setUsersByUsernameQuery` et `setAuthoritiesByUsernameQuery` en interrogeant vos tables `usuario` et `rol` via l'`email`.

- **Login personnalisé & Redirection intelligente :**
  - Configurez `.loginPage("/loginForm")` et `.loginProcessingUrl("/processLogin")`.
  - Implémentez un `AuthenticationSuccessHandler` : récupérez l'entité `Usuario` complète via `UsuarioRepository.findByEmail(...)` et enregistrez-la en session avec `session.setAttribute("usuario", usuario)`. *(Rappel : la classe entité `Usuario` doit implémenter `java.io.Serializable`).*
  - Vérifiez la présence de l'attribut de session `SPRING_SECURITY_SAVED_REQUEST` (`DefaultSavedRequest`) afin de rediriger l'utilisateur vers l'URL protégée initialement demandée. À défaut, redirigez un `admin` vers `/proveedores` et un utilisateur `logistica` vers `/productos`.
- **Déconnexion & Persistance de session :** Activez `spring.session.store-type=jdbc` dans `application.properties`. Configurez le `logout()` pour invalider la session HTTP (`invalidateHttpSession(true)`), supprimer le cookie `JSESSIONID` et rediriger vers `/catalogo`.

1.3 Sécurisation de la Vue Thymeleaf (`navbar.html`)

Intégrez le namespace `xmlns:sec="http://www.thymeleaf.org/extras/spring-security"` dans votre barre de navigation :

- Affichez le bouton *« Iniciar Sesión »* uniquement aux utilisateurs non connectés via `sec:authorize="isAnonymous()"`.
- Affichez le message `Bienvenido [Nombre] [Apellido]` en lisant `${session.usuario}` et le formulaire POST `/logout` uniquement lorsque `sec:authorize="isAuthenticated()"`.
- Conditionnez l'affichage des liens de navigation vers `/proveedores` et `/productos` à l'aide de `sec:authorize="hasAnyAuthority(...)"`.

Partie 2 : Création du Web Service RESTful (`api-inventario`)

Dans `api-inventario` (port `8081`), développez un `@RestController` mappé sur `/api/productos` renvoyant des `ResponseEntity<HashMap<String, Object>>` ainsi qu'un contrôleur `/api/proveedores` (`GET` pour lister les fournisseurs).

Méthode	Endpoint	Spécifications fonctionnelles et codes HTTP attendus
GET	`/api/productos`	Retourne la liste complète des produits en JSON (HTTP `200 OK`). Utilisez `@JsonIgnoreProperties` sur l'entité `Proveedor` si nécessaire pour maîtriser les champs sérialisés.
GET	`/api/productos/{id}`	Reçoit `id` au format `String`. Si l'ID n'est pas numérique (`NumberFormatException`) ou n'existe pas en base, retourne HTTP `400 Bad Request` avec `{"error": "..."}`. Sinon, retourne HTTP `200 OK` avec `{"estado": "encontrado", "producto": ...}`.

Méthode	Endpoint	Spécifications fonctionnelles et codes HTTP attendus
POST	`/api/productos`	Reçoit un objet `Producto` via `@RequestBody` et un paramètre optionnel `@RequestParam(required = false) boolean fetchId`. Sauvegarde l'entité et renvoie HTTP `201 Created` avec `{"estado": "creado"}` (en ajoutant `"id": <id_généré>` si `fetchId=true`).
PUT	`/api/productos`	**Mise à jour partielle :** Vérifie que `idproducto > 0` et que le produit existe en base. Ne met à jour que les champs non nuls envoyés dans le JSON (ne jamais écraser le nom ou le stock existants par `null` ou `0` si seul le prix est transmis). Retourne HTTP `200 OK` ou `400 Bad Request`.
DELETE	`/api/productos/{id}`	Valide que l'ID est numérique et présent en base, supprime le produit et retourne HTTP `200 OK` avec `{"estado": "eliminado"}` (ou HTTP `400` en cas d'échec).

Gestion globale des erreurs JSON : Ajoutez dans le contrôleur REST une méthode annotée avec `@ExceptionHandler(HttpMessageNotReadableException.class)` pour intercepter les requêtes `POST` ou `PUT` envoyées sans corps JSON (ou avec une syntaxe invalide) et renvoyer proprement un code HTTP `400 Bad Request` accompagné d'un message JSON explicite.

Partie 3 : Consommation de Services Web (`app-logistica`)

3.1 Intégration avec l'API Interne (`ProductoRestDao`)

Créez une classe de service annotée avec `@Service` utilisant `RestTemplate` pour connecter les contrôleurs MVC de `app-logistica` aux endpoints de `http://localhost:8081/api/productos` :

- **Lecture :** Utilisez `restTemplate.getForEntity(...)` pour récupérer la liste des produits et le détail d'un produit lors de l'édition.
- **Création et Mise à jour :** Construisez un `HttpHeaders` avec `MediaType.APPLICATION_JSON`, encapsulez l'objet dans un `HttpEntity<Producto>` et exécutez l'envoi via `restTemplate.postForEntity(...)` (création) ou `restTemplate.exchange(..., HttpMethod.PUT, ...)` (édition).
- **Suppression :** Appelez `restTemplate.delete(...)` sur l'URL ciblant l'identifiant du produit.

3.2 Consommation d'une API Externe avec Token Bearer (Validation RUC)

Dans le formulaire de création d'un fournisseur (`/proveedores/nuevo`), implémentez une fonctionnalité permettant de valider un numéro RUC (11 chiffres) auprès d'un service web externe (type `api.ateneaperu.com` ou un mock Postman) :

```
HttpHeaders headers = new HttpHeaders();
headers.set("Authorization", "Bearer " + apiToken);
headers.setContentType(MediaType.APPLICATION_JSON);
```

Envoyez la requête `POST` avec le paramètre `RUC` et le header d'autorisation ci-dessus. Désérialisez la réponse JSON (via un DTO dédié ou un `HashMap`) afin de récupérer automatiquement la raison sociale et de la pré-remplir dans la vue Thymeleaf.

Grille de Validation & Tests Attendus

Module	Scénario de test	Comportement attendu
Autorisation	Connexion avec le rôle `logistica` puis accès manuel à `/proveedores`.	Blocage par Spring Security (erreur HTTP `403 Forbidden`).
SavedRequest	Utilisateur anonyme accédant à `/productos/nuevo`, puis connexion réussie.	Redirection automatique vers `/productos/nuevo` (et non vers la route par défaut du rôle).
API REST (POST)	Requête `POST /api/productos` dans Postman avec un body vide.	Réponse HTTP `400 Bad Request` en JSON gérée par `@ExceptionHandler`.
API REST (PUT)	Requête `PUT /api/productos` avec `{"idproducto": 1, "precio": 45.50}`.	Seul le prix est mis à jour en base ; le nom et le stock existants restent intacts.
Résilience Client	Arrêt du serveur `api-inventario` (port 8081) et ouverture de `/productos` sur l'app web.	Capture de `ResourceAccessException` et affichage d'un message d'alerte propre dans la vue.